

Exercise for MA-INF 2213 Computer Vision SS25

29.04.2026

Submission deadline: 20.05.2026

Important: We recommend using **Python 3.12** for implementing your solutions in this exercise. You are **not** allowed to use any external Python libraries for implementing the Decision Tree or the Random Forest model in this exercise, other than the **NumPy (recommended version 1.26.4)** library. Otherwise, you will not get any points in all related parts. You need to use the **OpenML** library to download the data.

Grading: Submissions that generate runtime errors or produce obviously rubbish results (e.g., nans, inf, or meaningless visual output in future exercises) will receive at most 50% of the points. Add a *Answer.md* file along with your code submission with answers to any written questions.

Plagiarism: Plagiarism in any form is prohibited. If your solution contains code copied from any source (e.g., other students, or solutions from the web), you will receive **0 points** for the entire exercise sheet.

Submission: You can complete the exercise in groups of two, but only one submission per group is allowed. Include a *ReadMe.txt* file with the names of your group members. Points for solutions without a *ReadMe.txt* file will only be given to the uploader.

Random Forest Classifier

In this exercise, the aim is to implement a decision tree and a random forest using the NumPy library and train them on a real-world tabular dataset. The goal is to predict whether a flight will be delayed based on scheduling and route-related information. The dataset used in this assignment is the **Airlines dataset**, available via OpenML. It contains 8 input features (a mix of categorical and numerical variables) and a binary target variable.

Dataset Description

The dataset can be loaded using the OpenML API. The classification task is to predict whether a given flight will be delayed (target variable `Delay`).

Dataset characteristics:

- Number of instances: ~27,000
- Number of features: 8
- Feature types:
 - 6 categorical features (e.g., `Airline`, `Flight`, `AirportFrom`, `AirportTo`)
 - 2 numerical features (e.g., `Time`)

Important: The dataset contains a mix of **numerical and categorical features**, including some with high cardinality (e.g., `Flight`, `Airport`). You must decide how to preprocess these features appropriately before training your models. Briefly describe how you preprocess these features in the *Answer.md* file (*max 2–3 sentences*).

Note on Reproducibility

To ensure reproducibility and a consistent evaluation protocol across all submissions, please use the following code at the top of your solution:

```
1 import numpy as np
2 import random
3 import os
4 import openml
5
6 seed = 10
7
8 random.seed(seed)
9 os.environ['PYTHONHASHSEED'] = str(seed)
10 np.random.seed(seed)
11
12 # Load dataset
13 air = openml.datasets.get_dataset(42493)
14 X_air, y_air, _, _ = air.get_data(target=air.default_target_attribute)
15
16 # Convert to NumPy arrays if needed
17 X_air = np.array(X_air)
18 y_air = np.array(y_air)
19
20 # Fixed train/test split
21 n = X_air.shape[0]
22 indices = np.random.permutation(n)
23
24 split = int(0.8 * n)
25 train_idx, test_idx = indices[:split], indices[split:]
26
27 X_train, X_test = X_air[train_idx], X_air[test_idx]
28 y_train, y_test = y_air[train_idx], y_air[test_idx]
```

1 Implementing and Optimizing a Decision Tree

1.1 Implementation

Implement a decision tree classifier using only the NumPy library. Train the decision tree on the training set and evaluate its performance on the test set using classification accuracy. (5 points)

1.2 Optimization of Hyperparameters

Optimize the hyperparameters of your decision tree, such as:

- Maximum tree depth
- Minimum number of samples required to split

Try to achieve a test set accuracy of **62% or higher**. Comment on the impact of hyperparameter choices in the *Answer.md* file (max 3–5 sentences). (3 points)

2 Implementing and Optimizing a Random Forest

2.1 Implementation

Implement a random forest classifier using your decision tree implementation (NumPy only). Each tree should be trained on a bootstrap sample of the training data, and feature subsampling should be applied at each split. Use the information gain criterion for splitting nodes. Train the random forest on the training set and evaluate it on the test set. (7 points)

2.2 Optimization of Hyperparameters

Optimize the hyperparameters of your random forest, such as:

- Number of trees
- Maximum tree depth
- Number of features considered at each split

Try to achieve a test set accuracy of at least **64% or higher**. Compare the performance of the decision tree and the random forest. Discuss the effect of hyperparameters and the role of randomness (bootstrapping, feature subsampling) in the *Answer.md* file (max 5–7 sentences). (5 points)

Bonus: The top three submissions achieving the highest accuracy on the random forest will receive 2 bonus points.

Submission Requirements

Important: Your final submission must execute only a single training and evaluation run using your **best hyperparameters**. In previous years, some submissions included loops over multiple hyperparameter configurations (e.g., grid search or ablations), leading to excessive runtimes. To avoid this:

- Any hyperparameter search or experimental code must be **commented out**
- Only the **final selected configuration** should be active and executed

Submissions are evaluated under time constraints, and code that takes too long to run cannot be graded. You are still expected to perform extensive experimentation during development. The results of these experiments must be summarized and discussed in your *Answer.md* file, as specified earlier.